U10 · Bases de datos relacionales

En la unidad 8 guardaste datos en ficheros. Funciona, pero en cuanto necesitas **buscar, filtrar, relacionar o modificar** esos datos con garantías, una base de datos es la herramienta adecuada. Esta unidad es sobre cómo hablar con ella desde Java.

1. Repaso exprés de SQL (con MySQL)

Un **SGBD relacional** (MySQL, MariaDB, PostgreSQL, Oracle…) guarda los datos en **tablas** (filas y columnas) y se maneja con **SQL**. Lo que vas a necesitar desde Java se resume en tres familias de sentencias:

```
-- DDL: define la estructura
CREATE DATABASE empresa;
USE empresa;
CREATE TABLE empleado (
    id        INT PRIMARY KEY AUTO_INCREMENT,
    nombre    VARCHAR(50) NOT NULL,
    categoria VARCHAR(30),
    salario   DECIMAL(8,2) DEFAULT 1000
);

-- DML: manipula los datos
INSERT INTO empleado (nombre, categoria, salario) VALUES ('Ana Pérez', 'Peón', 1400);
UPDATE empleado SET salario = 1500 WHERE id = 1;
DELETE FROM empleado WHERE id = 1;

-- Consultas
SELECT nombre, salario FROM empleado WHERE salario > 1200 ORDER BY salario DESC;
SELECT categoria, AVG(salario) FROM empleado GROUP BY categoria;
```

> **Practica el SQL primero fuera de Java**
>
> Antes de meter una sentencia en tu código, pruébala en un cliente de la base de datos (MySQL Workbench, la línea de comandos de MySQL…). Si la consulta falla, es mucho más fácil averiguar por qué sin Java por medio. Una vez que funciona, copiarla a tu programa es trivial.

2. JDBC y el driver

JDBC (*Java Database Connectivity*) es la API estándar de Java para hablar con bases de datos relacionales. Vive en el paquete `java.sql` (y `javax.sql` para las partes más avanzadas) y tiene una idea clave: Java solo define **interfaces** (`Connection`, `Statement`, `ResultSet`…) y cada fabricante de base de datos entrega un **driver** que las implementa. Por eso el código es casi el mismo para MySQL que para PostgreSQL u Oracle: cambia el driver y la URL de conexión, poco más.

Hay cuatro tipos de driver; el que usarás es el **tipo 4**, escrito íntegramente en Java y que habla directamente el protocolo de la base de datos, sin instalar nada en el cliente. Para MySQL es **MySQL Connector/J**.

Añadir el driver a tu proyecto en IntelliJ

El driver es un fichero `.jar` que tu proyecto necesita en el *classpath*. Lo más cómodo es añadirlo desde Maven:

- **Con un proyecto Maven o Gradle**: añade la dependencia `com.mysql:mysql-connector-j` en tu `pom.xml` o `build.gradle` y recarga el proyecto.
- **Sin gestor de dependencias**: *File → Project Structure → Libraries → + → From Maven…*, busca `com.mysql:mysql-connector-j` y acéptalo.

> **Si tu proyecto usa módulos**
>
> Con un fichero `module-info.java`, añade `requires java.sql;` para poder usar `java.sql.*`.

Antiguamente había que cargar el driver a mano con `Class.forName("com.mysql.cj.jdbc.Driver")`. Desde JDBC 4 **ya no hace falta**:

`DriverManager` encuentra solo los drivers que estén en el classpath. Lo verás en muchos ejemplos antiguos; ahora es innecesario.

Modelos de dos y tres capas

- **Dos capas**: tu aplicación se conecta directamente al servidor de base de datos (lo que haremos en esta unidad).
- **Tres capas**: la aplicación habla con un servicio intermedio (un servidor de aplicaciones) y es **ese** servicio quien habla con la base de datos. Es lo habitual en aplicaciones web y móviles, porque el cliente nunca recibe la contraseña de la base de datos.

3. Conectar y ejecutar sentencias

Las cuatro piezas que necesitas son siempre las mismas: `DriverManager` → `Connection` → `Statement` → `ResultSet`.

```
String url = "jdbc:mysql://localhost:3306/empresa";

try (Connection con = DriverManager.getConnection(url, "usuario", "contraseña");
     Statement st = con.createStatement();
     ResultSet rs = st.executeQuery("SELECT id, nombre, salario FROM empleado")) {

    while (rs.next()) {
        int id = rs.getInt("id");
        String nombre = rs.getString("nombre");
        double salario = rs.getDouble("salario");
        System.out.println(id + " " + nombre + " " + salario);
    }
} catch (SQLException e) {
    System.err.println("Error de base de datos: " + e.getMessage());
}
```

- La **URL** tiene la forma `jdbc:mysql://servidor:puerto/nombreBD`. Si falla la conexión (URL mal escrita, servidor apagado, usuario o contraseña incorrectos, base de datos inexistente) se lanza una `SQLException`.
- Las tres clases son `AutoCloseable`, así que **`try-with-resources`** las cierra solas, en orden inverso, aunque haya un error. Los ejemplos antiguos cierran a mano con `close()` en un

`finally`; con `try-with-resources` ya no hace falta, y es mucho menos propenso a olvidos.

- Un `ResultSet` tiene un **cursor** que empieza **antes** de la primera fila: `next()` lo avanza y devuelve `false` cuando ya no quedan filas. Las columnas se pueden leer por nombre (`getString("nombre")`) o por posición (`getString(2)`), **empezando en 1**, no en 0.

Tres formas de ejecutar una sentencia

Método	Para qué	Devuelve
`executeQuery(sql)`	`SELECT`	Un `ResultSet` con las filas
`executeUpdate(sql)`	`INSERT`, `UPDATE`, `DELETE`	Un `int`: cuántas filas se han visto afectadas
`execute(sql)`	Cualquiera (se usa sobre todo para DDL: `CREATE TABLE`…)	Un `boolean`: `true` si el resultado es un `ResultSet`

```
int filas = st.executeUpdate("UPDATE empleado SET salario = salario * 1.05 WHERE categoria = 'Peón'");
System.out.println(filas + " empleados actualizados");
```

4. Sentencias preparadas: `PreparedStatement`

Casi nunca vas a escribir la sentencia con valores fijos: los datos vienen de un formulario, de un fichero, de otro método. La tentación es **concatenar**:

```
// ¡NUNCA hagas esto!
String sql = "SELECT * FROM usuarios WHERE usuario = '" + usuario + "' AND clave = '" + clave + "'";
```

> ⚠ **Inyección SQL**
>
> Si el usuario escribe como nombre `' OR '1'='1`, la sentencia anterior se convierte en `... WHERE usuario = '' OR '1'='1' AND ...` y **deja entrar a cualquiera**. Un atacante podría incluso añadir un `; DROP TABLE usuarios;`. La inyección SQL es una de las vulnerabilidades más antiguas y más explotadas de internet, y se evita siempre de la misma forma: **nunca concatenes datos en una sentencia SQL**.

La solución es `PreparedStatement`: escribes la sentencia con **marcadores `?`** y le pasas los valores aparte. El driver los trata como datos, jamás como código SQL:

```
String sql = "SELECT nombre FROM usuarios WHERE usuario = ? AND clave = ?";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, usuario);        // el primer ? (la posición empieza en 1)
    ps.setString(2, clave);
    try (ResultSet rs = ps.executeQuery()) {
        boolean valido = rs.next();   // true si hay al menos una fila
    }
}
```

Además de seguro, es más cómodo (no hay que escapar comillas ni formatear fechas) y más eficiente cuando repites la misma sentencia con valores distintos. Hay un `setXxx` para cada tipo: `setInt`, `setDouble`, `setString`, `setDate`, `setBoolean`…

Insertar y recuperar la clave generada

Cuando una tabla tiene una clave `AUTO_INCREMENT`, suele interesarte saber qué `id` se le ha asignado a la fila recién insertada:

```
String sql = "INSERT INTO empleado (nombre, categoria, salario) VALUES (?, ?,
?)";
try (PreparedStatement ps = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, "Ana Pérez");
    ps.setString(2, "Peón");
    ps.setDouble(3, 1400);
    ps.executeUpdate();

    try (ResultSet claves = ps.getGeneratedKeys()) {
        if (claves.next()) {
            System.out.println("Nuevo id: " + claves.getInt(1));
        }
    }
}
```

5. Trabajar con `ResultSet`

Tipos de `ResultSet`

Por defecto, un `ResultSet` solo se recorre **hacia delante** y es **de solo lectura**. Se puede pedir otro comportamiento al crear el `Statement`:

```
Statement st = con.createStatement(
        ResultSet.TYPE_SCROLL_INSENSITIVE,   // se puede mover libremente (no refleja
cambios hechos por otros)
        ResultSet.CONCUR_UPDATABLE);         // y se puede modificar
```

Constante	Significado
`TYPE_FORWARD_ONLY`	Solo `next()`. Es el valor por defecto
`TYPE_SCROLL_INSENSITIVE`	Cursor libre (`first()`, `last()`, `previous()`, `absolute(n)`…), sin ver los cambios de otros
`TYPE_SCROLL_SENSITIVE`	Cursor libre y refleja los cambios que otros hagan en la base de datos

Constante	Significado
`CONCUR_READ_ONLY` / `CONCUR_UPDATABLE`	Solo lectura / se puede modificar con `updateXxx` + `updateRow()`

En la práctica, lo más habitual es el `ResultSet` por defecto: recorrer una vez, copiar lo necesario a objetos Java, y listo.

Conversión de tipos entre SQL y Java

SQL (MySQL)	Java	Método
`INT`	`int`	`getInt`
`BIGINT`	`long`	`getLong`
`DECIMAL`, `DOUBLE`	`double` / `BigDecimal`	`getDouble` / `getBigDecimal`
`VARCHAR`, `TEXT`	`String`	`getString`
`DATE`	`java.sql.Date` → `LocalDate`	`getDate(...).toLocalDate()`
`TIMESTAMP`	`java.sql.Timestamp` → `LocalDateTime`	`getTimestamp(...).toLocalDateTime()`
`BOOLEAN`	`boolean`	`getBoolean`

> ⚠ **Cuidado con los `NULL`**
>
> Si una columna numérica contiene `NULL`, `getInt` devuelve `0` (no un error), y no hay forma de distinguirlo de un 0 real. Si importa, llama después a `rs.wasNull()` para saber si el último valor leído era `NULL`.

6. Transacciones

Una sentencia SQL individual es **atómica**: o se hace entera o no se hace. Una **transacción** extiende esa garantía a **varias sentencias**: o se aplican todas, o ninguna.

El ejemplo clásico es una transferencia bancaria: restar de una cuenta y sumar en otra. Si el programa falla entre las dos operaciones, el dinero habría desaparecido.

Por defecto, JDBC trabaja en modo **autocommit**: cada sentencia se confirma sola, inmediatamente. Para usar transacciones hay que desactivarlo y confirmar (o deshacer) a mano:

```
try (Connection con = DriverManager.getConnection(url, usuario, clave)) {
    con.setAutoCommit(false);            // empieza la transacción
    try (PreparedStatement quitar = con.prepareStatement("UPDATE cuenta SET saldo =
saldo - ? WHERE id = ?");
         PreparedStatement poner = con.prepareStatement("UPDATE cuenta SET saldo =
saldo + ? WHERE id = ?")) {

        quitar.setDouble(1, 100);  quitar.setInt(2, 1);  quitar.executeUpdate();
        poner.setDouble(1, 100);   poner.setInt(2, 2);   poner.executeUpdate();

        con.commit();                    // ambas se hacen permanentes a la vez
    } catch (SQLException e) {
        con.rollback();                  // algo falló: se deshace TODO lo hecho
desde el inicio
        throw e;
    }
}
```

7. Procedimientos almacenados y datos binarios

Un **procedimiento almacenado** es código SQL guardado dentro del propio servidor, que se ejecuta con `CallableStatement`:

```
try (CallableStatement cs = con.prepareCall("{call subir_salario(?, ?)}")) {
    cs.setString(1, "Peón");
    cs.setDouble(2, 5.0);
    cs.execute();
}
```

Para parámetros de salida se declaran antes con `cs.registerOutParameter(posicion, tipo)` y se leen después con `cs.getXxx(posicion)`.

También puedes guardar **datos binarios** (imágenes, documentos PDF) en columnas de tipo `BLOB`, pasándolos como un flujo de bytes:

```
try (PreparedStatement ps = con.prepareStatement("INSERT INTO foto (nombre,
imagen) VALUES (?, ?)");
     InputStream in = new FileInputStream("perfil.png")) {
    ps.setString(1, "perfil");
    ps.setBinaryStream(2, in);      // la imagen viaja como un flujo de bytes
    ps.executeUpdate();
}
```

8. Metadatos y pools de conexiones

- **Metadatos**: JDBC puede describirte la propia base de datos (`con.getMetaData()`: nombre del SGBD, lista de tablas, si soporta transacciones…) y la forma de un resultado (`rs.getMetaData()`: cuántas columnas tiene, cómo se llaman, de qué tipo). Es lo que permite escribir código genérico, como una función que muestre *cualquier* consulta como una tabla sin saber de antemano sus columnas.
- **`DataSource` y *pool* de conexiones**: abrir una conexión es lento. En una aplicación real (sobre todo una web), en vez de abrir y cerrar una conexión por operación, se mantiene un **pool**: un conjunto de conexiones ya abiertas que se prestan y se devuelven. Para ello se usa un `DataSource` (de `javax.sql`) proporcionado por una librería como HikariCP. Para esta unidad basta con que conozcas la idea.

9. Mostrar una consulta en una ventana

Aquí se juntan dos unidades: el contenido de un `ResultSet` se vuelca en un `DefaultTableModel` y se asigna a un `JTable` (unidad 9):

```
DefaultTableModel modelo = new DefaultTableModel();
modelo.addColumn("Id");
modelo.addColumn("Nombre");
modelo.addColumn("Salario");

try (Connection con = DriverManager.getConnection(url, usuario, clave);
     Statement st = con.createStatement();
     ResultSet rs = st.executeQuery("SELECT id, nombre, salario FROM empleado")) {
    while (rs.next()) {
        modelo.addRow(new Object[]{rs.getInt(1), rs.getString(2), rs.getDouble(3)});
    }
}
tabla.setModel(modelo);
```

> ⚠ **Y no desde el hilo de eventos**
>
> Recuerda lo que vimos en la unidad 9: el código de un listener se ejecuta en el hilo de eventos de Swing. Una consulta que tarde unos segundos (red lenta, tabla grande) **congela la ventana**. Para una práctica pequeña con una base de datos local no se nota, pero en una aplicación real las consultas se hacen en otro hilo (por ejemplo con un `SwingWorker`).

10. Persistencia de objetos: el patrón DAO

Mezclar SQL con el resto del programa lo hace imposible de mantener. El patrón **DAO** (*Data Access Object*) separa el acceso a datos de la lógica de la aplicación: **una clase por entidad** que concentra *todas* las operaciones con la base de datos, y que devuelve y recibe objetos Java normales (a estos objetos "solo datos" se les llama **POJO**).

```java
public class Empleado {
    private int id;
    private String nombre;
    private String categoria;
    private double salario;
    // constructor, getters y setters
}

public class EmpleadoDao {
    private final Connection con;

    public EmpleadoDao(Connection con) {
        this.con = con;
    }

    public void insertar(Empleado e) throws SQLException {
        String sql = "INSERT INTO empleado (nombre, categoria, salario) VALUES (?, ?,
?)";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, e.getNombre());
            ps.setString(2, e.getCategoria());
            ps.setDouble(3, e.getSalario());
            ps.executeUpdate();
        }
    }

    public List<Empleado> buscarTodos() throws SQLException {
        List<Empleado> lista = new ArrayList<>();
        try (PreparedStatement ps = con.prepareStatement("SELECT id, nombre,
categoria, salario FROM empleado");
             ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                lista.add(new Empleado(rs.getInt("id"), rs.getString("nombre"),
                                       rs.getString("categoria"),
rs.getDouble("salario")));
            }
        }
        return lista;
    }
    // actualizar(...), borrar(...), buscarPorId(...)
```

```
}
```

El resto de la aplicación solo conoce `EmpleadoDao` y `Empleado`: no sabe qué base de datos hay detrás, ni que existe SQL. Si mañana cambias de MySQL a PostgreSQL (o a un fichero), solo reescribes el DAO. Es la misma idea de separación de capas que MVC en la unidad 9, aplicada a los datos.

> **¿Y el Singleton para la conexión?**
>
> Es habitual ver el DAO con una conexión única compartida mediante el patrón Singleton (unidad 4). Funciona en un programa sencillo de un solo hilo, pero una única `Connection` **no es segura para varios hilos** y, si se cae, toda la aplicación se queda sin base de datos. En aplicaciones serias se sustituye por un *pool* de conexiones.

11. Mapeo objeto-relacional (ORM) y JPA

Los objetos y las tablas no se corresponden exactamente: una clase tiene referencias a otros objetos, herencia, listas; una tabla tiene claves ajenas. A esa diferencia se le llama **desfase objeto-relacional**. Con JDBC puro, el "traductor" lo escribes a mano (`setString(1, e.getNombre())`, `rs.getString("nombre")`…) para cada clase.

Un **ORM** (*Object-Relational Mapping*) automatiza esa traducción. El estándar de Java es **JPA** (*Jakarta Persistence API*), y **Hibernate** y **EclipseLink** son implementaciones. En vez de escribir SQL, **anotas** la clase para decir con qué tabla se corresponde:

```
@Entity
@Table(name = "empleado")
public class Empleado {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String nombre;
    private String categoria;
    private double salario;
    // ...
}
```

Y guardar un objeto pasa a ser una línea, sin una sola sentencia SQL:

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
em.persist(new Empleado("Ana Pérez", "Peón", 1400));   // INSERT INTO empleado ...
em.getTransaction().commit();
```

Las dos formas tienen sentido: JDBC te da control total y es la base que debes dominar; JPA te ahorra mucho código repetitivo en aplicaciones grandes, pero por debajo sigue usando JDBC.

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA9. Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y consistencia de los datos.	a) características y métodos de acceso a SGBD relacionales · b) conexiones con bases de datos · c) código para almacenar información · d) programas para recuperar y mostrar información · e) borrados y modificaciones · f) aplicaciones que ejecutan consultas · g) aplicaciones para gestionar la información de la base de datos

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras